

claude-windows / 96963dab-d38f-488c-af44-530ddbefe1

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbefe1\subagents\workflows\wf_dd531439-5e0\agent-a6bd58d9cd91ec0f9.jsonl`  
- SHA-256: `f285c9fd34793b43806631a91bb64e0cac77291a6ab3bbcf6fc7789cf5d734de`  
- Source modified: `2026-07-08T06:33:02+00:00`  
- Imported at: `2026-07-08T15:59:27+00:00`  
- project: `wf_dd531439-5e0`  
- session_id: `96963dab-d38f-488c-af44-530ddbefe1`
```

Transcript

1. user (2026-07-08T06:26:58.706Z)

You are reviewing an UNCOMMITTED diff in an isolated git worktree at C:/Users/avidu/AppData/Local/Temp/claude/wt524-969 (branch feat/524-l4-primary, based on origin/master).

To see the change: cd to the worktree and run `git --no-pager diff` (working tree vs HEAD) and `git --no-pager diff --stat`; read the new files docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md and tests/test_video_id_trust.py directly. Read surrounding code as needed ? do NOT limit yourself to diff hunks.

CONTEXT ? GitHub issue #524 (epic): collapse the pipeline onto the L4 worker; shrink control-plane to a thin router. This PR delivers (1) a design doc resolving the compute-topology question (recommend co-located scale-to-zero Cloud Run Jobs + bucket upload; REJECT upload-onto-L4 and persistent-GPU with a cost model), and (2) "win #2": the control-plane drain trusts the SERVER-DERIVED submit-time md5 (jobs.video_id ? submit_time_video_id, #484 ? GCS metadata md5 converted to hex by backend/storage/gcs.py::_md5_hex) instead of re-hashing the fetched clip, gated by deployment.trust_submit_video_id (default True) and guarded by orchestration._looks_like_video_id (strict 32-char lowercase hex). Win #1 (validation-on-L4) is deliberately NOT in this PR ? it is delivered by the separate open branch feat/521-compile-on-l4 (phase_placement.validation); duplicating it is forbidden.

Report ONLY concrete, actionable findings with file:line. No praise, no restating the design. For each: what breaks, the exact failing input/state, and the fix. If a dimension is clean, say so explicitly.

DIMENSION: REPO CONVENTIONS / DISCIPLINE. Check against CLAUDE.md + the map:

- Config knob: is deployment.trust_submit_video_id placed + documented idiomatically (mirrors compute_target/remote_poll_max_wait_sec)? Do the two config-shape tests (test_config_deployment.py) now include it? Is there a presets.py parity test or other shape-pinning test that ALSO needs updating but was missed? (grep the repo for every assertion that enumerates the deployment section.)
- Does the new field risk a merge conflict with #521 (which also edits DeploymentConfig)? Is it a non-overlapping addition?
- Tests (tests/test_video_id_trust.py): do they follow the repo's fully-mocked offline convention? Do they actually PROVE the win (assert compute_video_id NOT called on trust, called on fallback)? Any missing case (flag off, malformed, wiring None)?
- Docstrings/comments: match the surrounding density/idiom? Any CLAUDE.md guardrail touched (paid-LLM, licensing, minors) ? should be none, confirm.
- Did the PR avoid staging stray files (git add -A) and stay scoped? Any doc that MUST be updated per convention (DOC_STATUS, docs/README, NEXT_STEPS) that was skipped ? is skipping justified (this is not a terminal-state archival flip)?

2. assistant (2026-07-08T06:27:01.277Z)

I'll start by examining the diff in the worktree.

3. user (2026-07-08T06:27:03.616Z)

warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tests/test_video_id_trust.py', LF will be replaced by CRLF the next time Git touches it

backend/api/job_worker.py	5 +
backend/config/models.py	10 +
backend/orchestration.py	51 ++++
.../L4_PRIMARY_TOPOLOGY.md	225 ++++++
docs/COMPUTE_DECOUPLED_SERVING/README.md	2 +
tests/test_config_deployment.py	2 +
tests/test_video_id_trust.py	173 ++++++

7 files changed, 466 insertions(+), 2 deletions(-)

4. user (2026-07-08T06:27:05.015Z)

bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells (#522)

699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload (#518)

71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)

115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)

ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker (#515)

5. user (2026-07-08T06:27:09.917Z)

```
diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index a90b14e..f8da26b 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -173,6 +173,11 @@ def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
     db,
     req,
     progress=lambda stage: db.set_job_progress(job_id, stage),
+    # #524 win #2: hand the drain the submit-time md5 already on the job row
+    # (`submit_time_video_id`, #484 ? server-derived, byte-identical to
+    # compute_video_id) so it can skip re-hashing the fetched clip. "" ? unknown
+    # (local upload / non-gcs URI) ? the drain hashes as before.
+    trusted_video_id=job.get("video_id") or None,
 )
 if result.get("video_id"):
     db.set_job_video_id(job_id, result["video_id"])
diff --git a/backend/config/models.py b/backend/config/models.py
index a7c997c..71e43b8 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -481,6 +481,16 @@ class DeploymentConfig(BaseModel):
     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect + queue/cold-
     start,
     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid GPU run.
     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
+    # Trust the SUBMIT-TIME md5 instead of re-hashing the fetched file on the drain (#524 win
+    # #2).
+    # The chunked upload already streams the whole-file md5 (`backend/api/uploads.py`), and
+    the
+    # submit hook records it on the job row (`submit_time_video_id`, one gs:// metadata stat
+    ? #484);
+    # that value is BYTE-IDENTICAL to `compute_video_id` (same bytes, same MD5) and is
+    ALREADY
+    # trusted for the submit-time cache probe (`backend/api/jobs.py`). When True (default)
+    the drain
+    # reuses it and SKIPS re-hashing the multi-GB clip it just fetched to scratch; when the id
+    is absent
+    # (a local/appliance upload or a non-gcs direct-URI submit ? `submit_time_video_id`
+    returns None)
```

```

+ # the drain still hashes, exactly as before. Set False to force a re-hash unconditionally
+ # (belt-and-suspenders rollback ? reverts to pre-#524 behaviour with no redeploy).
+ trust_submit_video_id: bool = True

class BillingConfig(BaseModel):
diff --git a/backend/orchestration.py b/backend/orchestration.py
index 8b7851e..7769e4e 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -239,6 +239,20 @@ def submit_time_video_id(input_ref, cfg) -> Optional[str]:
    return None

+def _looks_like_video_id(value: Optional[str]) -> bool:
+    """True iff ``value`` is a well-formed ``video_id`` ? the 32-char lowercase hex MD5 that
+    ``compute_video_id`` produces (``backend/utils/hashing.py``). Guards the #524 win-#2 trust
+    path:
+    + a trusted id is only reused in place of a re-hash when it has the EXACT shape the drain
+    would
+    + have computed, so a malformed / truncated / wrong-encoding id can never become the DB key ?
+    + it degrades to a real hash instead. ``submit_time_video_id`` already normalizes the gs://
+    + base64 md5 to this hex form (#484, ``backend/storage/gcs.py::_md5_hex``)."""
+    return (
+        isinstance(value, str)
+        and len(value) == 32
+        and all(c in "0123456789abcdef" for c in value)
+    )
+
+def _cached_process_payload(
+    video_id: str, validation_results, segments, requested_compute
+) -> Dict[str, Any]:
@@ -706,7 +720,7 @@ def _maybe_dispatch_remote(

def _execute_processing(
-    config=None, db=None, req=None, progress=None
+    config=None, db=None, req=None, progress=None, trusted_video_id=None
) -> Dict[str, Any]:
    """The full hash ? validate ? segment ? report pipeline for one video.

@@ -716,6 +730,15 @@ def _execute_processing(
    test code that calls ``_execute_processing(req)`` still works.

    ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
+
+    ``trusted_video_id`` is the SERVER-DERIVED submit-time md5 the drain already knows for this
+    job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ? byte-identical to
+    ``compute_video_id`` and already trusted for the submit-time cache probe. When present (and
+    ``deployment.trust_submit_video_id`` is on), it is reused so the drain SKIPS re-hashing the
+    multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes as
+    before.
+
+    It is a keyword-only, internal channel ? NOT a client-supplied request field (a caller's
+    ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative id).
+
+    Raises on unexpected internal errors ? the caller records those as a job failure
    """
    import backend.main as _main # call-time seam: main's globals/model stay patchable
@@ -748,7 +771,31 @@ def _execute_processing(
    local_video = storage.acquire_local_input(
        req.video_path, getattr(config, "storage", None)
    )
-    video_id = _main.compute_video_id(local_video)
+    # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing the whole

```

```

+         # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
+         # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the submit-
time
+         # cache probe trusts, so reusing it here is consistent, not a new trust assumption.
Gated by
+         # deployment.trust_submit_video_id (default on) and only honoured when the id is well-
formed
+         # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The fetch
above
+         # is unchanged; only the redundant hash is skipped.
+         trust_submitted = bool(
+             getattr(
+                 getattr(config, "deployment", None), "trust_submit_video_id", True
+             )
+         )
+         # STRICT: trust only an id in the EXACT form the drain would have produced (32-char
+         # lowercase hex, per compute_video_id / submit_time_video_id). No normalization ? a
+         # differently-cased or malformed id is a signal something is off, so re-hash rather
than
+         # silently coerce it into the DB key.
+         candidate = trusted_video_id or ""
+         if trust_submitted and _looks_like_video_id(candidate):
+             video_id = candidate
+             logger.info(
+                 "[API] trusting submit-time video_id=%s ? skipped the drain re-hash (#524).",
+                 video_id,
+             )
+         else:
+             video_id = _main.compute_video_id(local_video)
except Exception:
    storage.cleanup_acquired_local_input(
        req.video_path, local_video, getattr(config, "storage", None)
diff --git a/tests/test_config_deployment.py b/tests/test_config_deployment.py
index 1e4cfa6..48150a2 100644
--- a/tests/test_config_deployment.py
+++ b/tests/test_config_deployment.py
@@ -70,6 +70,7 @@ def test_default_config_dump_only_adds_deployment_section():
    "compute_target": "",
    "max_concurrent_remote_jobs": 4,
    "remote_poll_max_wait_sec": 3900.0,
+    "trust_submit_video_id": True,
    }

@@ -397,6 +398,7 @@ def test_save_default_config_includes_deployment(tmp_path):
    "compute_target": "",
    "max_concurrent_remote_jobs": 4,
    "remote_poll_max_wait_sec": 3900.0,
+    "trust_submit_video_id": True,
    }
    cfg = load_config(str(p))
    assert cfg.deployment.profile == ""

```

6. assistant (2026-07-08T06:27:17.627Z)

Now let me read the test file and check for other places enumerating the deployment section.

7. user (2026-07-08T06:27:18.702Z)

```

1     ""#524 win #2 ? trust the submit-time ``video_id`` instead of re-hashing on the drain.
2
3     The chunked upload streams the whole-file MD5 while it uploads
(``backend/api/uploads.py``), and the
4     submit hook records that same md5 on the job row (``submit_time_video_id``, #484 ? one
gs:// metadata

```

```

5      stat, converted to the identical HEX form ``compute_video_id`` produces). So when the
drain fetches
6      the clip to scratch it already KNOWS the video_id and need not re-hash the (often multi-
GB) file.
7
8      These tests pin the behaviour: with a well-formed trusted id and the knob on (default),
the drain
9      SKIPS ``compute_video_id`` and uses the trusted id verbatim; with the id absent,
malformed, or the
10     knob off, it hashes exactly as before. A spy on ``compute_video_id`` (counting
invocations) is the
11     proof ? mirrors ``tests/test_validation_cache.py``'s stubbed-suite pattern (fully
offline, no decode).
12     """
13
14     import backend.main as m
15     from backend.config.models import AppConfig, DeploymentConfig
16     from backend.infrastructure.database import Database
17     from backend.orchestration import _looks_like_video_id
18     from backend.pipeline.validators.base import ValidationResult
19
20     _VALID_ID = "a" * 32 # a well-formed 32-char lowercase-hex video_id
21     _HASH_SENTINEL = "b" * 32 # what a real compute_video_id() would return (spied)
22
23
24     class _HashSpy:
25         """Stands in for ``compute_video_id`` and counts real invocations. A TRUST hit must
NOT call this
26         ? ``calls == 0`` is the proof the drain skipped the re-hash."""
27
28         def __init__(self, ret=_HASH_SENTINEL):
29             self._ret = ret
30             self.calls = 0
31
32         def __call__(self, *a, **k):
33             self.calls += 1
34             return self._ret
35
36
37     class _OkSeg:
38         """A segmenter that writes one play row (validation is stubbed to pass, so it's
reached)."""
39
40         def __init__(self, db_, cfg_):
41             self.db = db_
42
43         def process_video(self, vid, path, *a, **k):
44             sid = self.db.add_play_segment(vid, 0.0, 5.0)
45             return [
46                 {"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0, "metadata":
{}}
47             ]
48
49
50     def _pass_validators(*a, **k):
51         return [ValidationResult(validator_name="x", passed=True, message="ok",
details={})], {}
52
53
54     def _setup(tmp_path, monkeypatch, *, cfg=None):
55         """Wire module globals + a temp-file DB + a hash spy + stubbed validators/segmenter.
56         Returns ``(hash_spy, video_path)``. A bare/local ``video_path`` makes
``acquire_local_input`` an
57         identity (no fetch), so the ONLY video_id source is the trusted id or the spied
hash."""

```

```

58     db = Database(str(tmp_path / "t.db"))
59     monkeypatch.setattr(m, "db_instance", db)
60     monkeypatch.setattr(m, "global_config", cfg or AppConfig())
61     spy = _HashSpy()
62     monkeypatch.setattr(m, "compute_video_id", spy)
63     monkeypatch.setattr(m.registry, "run_all_with_context", _pass_validators)
64     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _OkSeg)
65     video = tmp_path / "v.mp4"
66     video.write_bytes(b"some-bytes")
67     return spy, str(video)
68
69
70     def _process(video, *, trusted):
71         # config resolves from m.global_config (set by _setup) via the backward-compat shim.
72         return m._execute_processing(
73             m.ProcessRequest(video_path=video, segmenter="motion"),
74             trusted_video_id=trusted,
75         )
76
77
78     # --- _looks_like_video_id (the format guard)
79     -----
80
81     def test_looks_like_video_id_accepts_only_32_char_lowercase_hex():
82         assert _looks_like_video_id("a" * 32)
83         assert _looks_like_video_id("0123456789abcdef0123456789abcdef")
84         # rejects: wrong length, uppercase, non-hex, non-str, empty, None
85         assert not _looks_like_video_id("a" * 31)
86         assert not _looks_like_video_id("a" * 33)
87         assert not _looks_like_video_id("A" * 32) # helper is strict-lowercase (callers
88         .lower() first)
89         assert not _looks_like_video_id("g" * 32)
90         assert not _looks_like_video_id("not-a-hash")
91         assert not _looks_like_video_id("")
92         assert not _looks_like_video_id(None)
93         assert not _looks_like_video_id(12345678901234567890123456789012) # int, not str
94
95     # --- the drain: trust vs re-hash
96     -----
97
98     def test_trusts_valid_submit_id_and_skips_rehash(tmp_path, monkeypatch, caplog):
99         """A well-formed trusted id + the default knob ? the drain reuses it and does NOT
100         call the hash."""
101         spy, video = _setup(tmp_path, monkeypatch)
102         with caplog.at_level("INFO", logger="backend.orchestration"):
103             out = _process(video, trusted=_VALID_ID)
104             assert out["video_id"] == _VALID_ID
105             assert spy.calls == 0 # the whole point: no re-hash
106             assert any("trusting submit-time video_id" in r.message for r in caplog.records)
107
108     def test_hashes_when_no_trusted_id(tmp_path, monkeypatch):
109         """No trusted id (local upload / non-gcs URI ? submit_time_video_id was None) ? hash
110         as before."""
111         spy, video = _setup(tmp_path, monkeypatch)
112         out = _process(video, trusted=None)
113         assert out["video_id"] == _HASH_SENTINEL
114         assert spy.calls == 1
115
116     def test_ignores_trusted_id_when_knob_off(tmp_path, monkeypatch):
117         """deployment.trust_submit_video_id=False forces the re-hash even with a valid

```

```

trusted id
118         (belt-and-suspenders rollback, no redeploy)."""
119         cfg = AppConfig(deployment=DeploymentConfig(trust_submit_video_id=False))
120         spy, video = _setup(tmp_path, monkeypatch, cfg=cfg)
121         out = _process(video, trusted=_VALID_ID)
122         assert out["video_id"] == _HASH_SENTINEL
123         assert spy.calls == 1
124
125
126     def test_ignores_malformed_trusted_id_and_falls_back_to_hash(tmp_path, monkeypatch):
127         """A malformed / wrong-length / wrong-encoding trusted id can never become the DB
key ?
128         it degrades to a real hash."""
129         spy, video = _setup(tmp_path, monkeypatch)
130         for bad in ("not-a-valid-id", "a" * 31, "A" * 32, ""):
131             spy.calls = 0
132             out = _process(video, trusted=bad)
133             assert out["video_id"] == _HASH_SENTINEL, bad
134             assert spy.calls == 1, bad
135
136
137     # --- the wiring: the drain hands _execute_processing the job row's video_id
-----
138
139
140     def test_run_claimed_job_threads_job_video_id_as_trusted(tmp_path, monkeypatch):
141         """`_run_claimed_job` must pass the job row's persisted video_id
(submit_time_video_id, #484) to
142         `_execute_processing` as `trusted_video_id` ? the wiring that makes win #2 fire
on the drain."""
143         db = Database(str(tmp_path / "jobs.db"))
144         monkeypatch.setattr(m, "db_instance", db)
145         monkeypatch.setattr(m, "global_config", AppConfig())
146         seen = {}
147
148         def _spy_exec(config, db_, req, progress=None, trusted_video_id=None):
149             seen["trusted"] = trusted_video_id
150             return {"status": "success", "video_id": _VALID_ID, "play_segments": []}
151
152         monkeypatch.setattr(m, "_execute_processing", _spy_exec)
153         db.create_job("pj", "gs://b/clip.mp4")
154         db.set_job_video_id("pj", _VALID_ID)
155         m._run_claimed_job(db.get_job("pj"), m.global_config, m.db_instance)
156         assert seen["trusted"] == _VALID_ID
157
158
159     def test_run_claimed_job_passes_none_when_job_has_no_video_id(tmp_path, monkeypatch):
160         """A job with no known video_id (local upload) threads None ? the drain hashes
(fallback)."""
161         db = Database(str(tmp_path / "jobs.db"))
162         monkeypatch.setattr(m, "db_instance", db)
163         monkeypatch.setattr(m, "global_config", AppConfig())
164         seen = {}
165
166         def _spy_exec(config, db_, req, progress=None, trusted_video_id=None):
167             seen["trusted"] = trusted_video_id
168             return {"status": "success", "video_id": "x", "play_segments": []}
169
170         monkeypatch.setattr(m, "_execute_processing", _spy_exec)
171         db.create_job("pj2", "/local/v.mp4") # no video_id set
172         m._run_claimed_job(db.get_job("pj2"), m.global_config, m.db_instance)
173         assert seen["trusted"] is None
174

```

```

backend/api/job_worker.py:35:# scales to deployment.max_concurrent_remote_jobs to overlap those.
This does NOT parallelize local
backend/api/job_worker.py:58:    ``deployment.compute_target=cloud_run``, OR the per-request
``compute='cloud'`` override path is
backend/api/job_worker.py:63:    if getattr(dep, "compute_target", "") == "cloud_run":
backend/api/job_worker.py:74:    may run ``deployment.max_concurrent_remote_jobs`` drain workers
so I/O-bound remote dispatch/poll
backend/api/job_worker.py:82:        return max(1, int(getattr(dep,
"max_concurrent_remote_jobs", 1) or 1))
backend/api/job_worker.py:365:        getattr(getattr(config, "deployment", None),
"compute_target", "") == "cloud_run"
backend/api/job_worker.py:393:    ``deployment.remote_poll_max_wait_sec`` budget a live dispatch
gets), then complete or
backend/api/job_worker.py:437:        getattr(getattr(config, "deployment", None),
"remote_poll_max_wait_sec", 3900.0)
backend/compute/base.py:8:The seam is **default-OFF**: only ``deployment.compute_target ==
"cloud_run"`` selects a
backend/compute/base.py:29:    ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile
wait. The message carries the
backend/compute/cloud_run.py:21:The dispatched container is forced to run **INLINE**
(``compute_target=local_gpu`` + native detector)
backend/compute/cloud_run.py:56:# (compute_target=local_gpu disarms the cmd_infer dispatch
guard; native = the L4 WASB runner.)
backend/compute/cloud_run.py:58:    "deployment.compute_target=local_gpu",
backend/compute/cloud_run.py:328:        otherwise make this poll spin the full
``remote_poll_max_wait_sec`` (~65 min) before failing
backend/compute/__init__.py:4:``ComputeBackend`` ONLY when a non-local
``deployment.compute_target`` is selected, and ``None``
backend/compute/__init__.py:50:    """Return the compute backend for
``config.deployment.compute_target``, or ``None`` to run
backend/compute/__init__.py:53:    Only ``compute_target == "cloud_run"`` returns a non-None
backend (a ``CloudRunCompute``). Its
backend/compute/__init__.py:60:    server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
backend/compute/__init__.py:65:        else getattr(getattr(config, "deployment", None),
"compute_target", "") or ""
backend/compute/__init__.py:74:    # deployment.remote_poll_max_wait_sec must outlast the worker
Job's --task-timeout so a
backend/compute/__init__.py:80:        getattr(config, "deployment", None),
"remote_poll_max_wait_sec", None
backend/config/loader.py:200:        ct = merged["deployment"].get("compute_target")
# An active profile with no explicit compute_target
backend/config/loader.py:203:
(e.g. config.json blanked it
backend/config/loader.py:205:        # active profile's compute_target (it stays
meaningful for the M2+ ComputeBackend).
backend/config/loader.py:206:        merged["deployment"]["compute_target"] = canonical
backend/config/loader.py:209:        "[CONFIG] deployment profile %r normally uses
compute_target=%r but "
backend/config/loader.py:216:        "[CONFIG] deployment profile ACTIVE: %s
(compute_target=%s)",
backend/config/loader.py:218:        merged["deployment"].get("compute_target"),
backend/config/loader.py:254:# local storage and the compute_target, so this stays the handful
of knobs a fresh, non-technical
backend/config/presets.py:30:# Each profile maps 1:1 to its canonical compute_target. Used to
apply the preset and to warn
backend/config/presets.py:31:# when config.json explicitly overrides compute_target into a state
inconsistent with the profile.
backend/config/presets.py:39:# that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
backend/config/presets.py:45:    "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend/config/presets.py:52:    "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
backend/config/presets.py:75:    "deployment": {"profile": "cpu-mock", "compute_target":
"cpu_mock"},

```



```

backend/main.py:268: # default (deployment.compute_target). Only honoured for these two
values; see _maybe_dispatch_remote.
backend/main.py:401: "compute_target": getattr(deployment, "compute_target", "") or
"",
backend/orchestration.py:200: if (getattr(deployment, "compute_target", "") or "") ==
"cloud_run":
backend/orchestration.py:448: """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path):
when `deployment.compute_target
backend/orchestration.py:948: # COMPUTE_DECOUPLED_SERVING M6 ? API: with
deployment.compute_target=cloud_run, run the heavy
backend/worker/__main__.py:177: container runs this SAME cmd_infer INLINE (compute_target
forced local_gpu) ? never re-dispatch.
backend/worker/__main__.py:250: # M6: a cloud compute_target DISPATCHES to a Cloud Run Job
instead of running in-process.
backend/worker/__main__.py:253: # dispatcher forces compute_target=local_gpu, so it never
recursively re-dispatches.
tests/test_api_cloud_dispatch.py:6:done-marker into a local "bucket". Cardinal invariant:
DEFAULT-OFF ? only compute_target=cloud_run
tests/test_api_cloud_dispatch.py:108: "deployment": {"compute_target": "cloud_run"},
tests/test_api_cloud_dispatch.py:181: "deployment.compute_target=local_gpu" in joined
tests/test_api_cloud_dispatch.py:385: """compute_target='' ? get_compute_backend returns None
? the in-process segmenter runs
tests/test_api_cloud_dispatch.py:390: cfg = AppConfig() # compute_target defaults to "" #
noqa: F841
tests/test_api_cloud_dispatch.py:494: """Server configured for cloud
(compute_target=cloud_run), but a per-request compute='local'
tests/test_api_cloud_dispatch.py:517: """Server default is LOCAL (compute_target=''), but a
per-request compute='cloud' forces a Cloud
tests/test_api_cloud_dispatch.py:525: "deployment": {"compute_target": ""}, # server
default = in-process
tests/test_api_cloud_dispatch.py:564: {"deployment": {"compute_target": ""}}
tests/test_api_cloud_dispatch.py:614: assert caps["deployment"]["compute_target"] ==
"cloud_run"
tests/test_api_cloud_dispatch.py:700: "deployment": {"compute_target": ""}, # server
default = in-process
tests/test_api_cloud_dispatch.py:825: cfg = AppConfig.model_validate({"deployment":
{"compute_target": ""}}) # noqa: F841
tests/test_api_cloud_dispatch.py:990: AppConfig.model_validate({"deployment":
{"compute_target": ""}}),
tests/test_api_cloud_dispatch.py:993: {"deployment": {"compute_target": ""}}
tests/test_compute_backend.py:5:?read flow is proven for $0. The cardinal invariant: **default-
OFF** ? only `compute_target=cloud_run`
tests/test_compute_backend.py:151:def _cfg(compute_target="", storage_backend="local"):
tests/test_compute_backend.py:154: "deployment": {"compute_target": compute_target},

```

9. assistant (2026-07-08T06:27:27.967Z)

Let me look for other shape-pinning assertions and check `main.py` capabilities dump and presets.

10. user (2026-07-08T06:27:32.883Z)

```

400: "profile": getattr(deployment, "profile", "") or "",
401: "compute_target": getattr(deployment, "compute_target", "") or "",

```

11. user (2026-07-08T06:27:33.582Z)

```

tests/test_compute_backend.py:1049: """CP-rebuild $4.3: deployment.remote_poll_max_wait_sec
drives the dispatch poll budget
tests/test_compute_backend.py:1064: cfg.deployment.remote_poll_max_wait_sec = 42.0
tests/test_drain_concurrency.py:6:`deployment.max_concurrent_remote_jobs`. The worker count is
fixed at first executor creation from
tests/test_drain_concurrency.py:33: {"deployment": {"compute_target": "cloud_run",
"max_concurrent_remote_jobs": 5}}

```

```

tests/test_drain_concurrency.py:38:         {"deployment": {"compute_target": "local_gpu",
"max_concurrent_remote_jobs": 5}}
tests/test_drain_concurrency.py:50:         "deployment": {"compute_target": "",
"max_concurrent_remote_jobs": 4},
tests/test_drain_concurrency.py:58:         {"deployment": {"compute_target": "",
"max_concurrent_remote_jobs": 4}}
tests/test_drain_concurrency.py:72:         "max_concurrent_remote_jobs": 6,
tests/test_drain_concurrency.py:90: def test_max_concurrent_remote_jobs_is_bounded():
tests/test_drain_concurrency.py:92:     AppConfig.model_validate({"deployment":
{"max_concurrent_remote_jobs": 0}}) # ge=1
tests/test_drain_concurrency.py:94:     AppConfig.model_validate({"deployment":
{"max_concurrent_remote_jobs": 999}}) # le=64
tests/test_restart_recovery.py:211:     cfg.deployment.remote_poll_max_wait_sec = 0.05 # tiny
budget for the test
Binary file tests/__pycache__/test_compute_backend.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_config_deployment.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_drain_concurrency.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_restart_recovery.cpython-313-pytest-9.0.3.pyc matches

```

12. user (2026-07-08T06:27:46.721Z)

```

70:         "compute_target": "",
191:         "deployment": {"profile": "cpu-mock", "compute_target": "cloud_run"},
210:         "deployment": {"profile": "cloud-serving", "compute_target": ""},
398:         "compute_target": "",

```

13. user (2026-07-08T06:27:49.164Z)

```

tests/test_api_cloud_dispatch.py
tests/test_compute_backend.py
tests/test_config_deployment.py
tests/test_deploy_manifests.py
tests/test_drain_concurrency.py
tests/test_profile_parity.py
tests/test_restart_recovery.py
deploy/cloudrun/gen_service_yaml.py
deploy/cloudrun/post_deploy_smoke.ps1
deploy/cloudrun/README.md
---golden config fixtures---

```

14. assistant (2026-07-08T06:27:52.106Z)

Let me check the profile parity test, which the prompt specifically flagged.

15. user (2026-07-08T06:27:52.431Z)

```

1      """Profile-parity ? the load-bearing decoupling proof (COMPUTE_DECOUPLED_SERVING M5,
L4).
2
3      The decoupling guarantee: the same `(trajectory + config)` produces the same candidate
4      windows + the same segment ranges** regardless of the active deployment profile /
orchestration.
5      Detection is replaced by the deterministic CPU stub (no GPU/WSL/network), so ANY
divergence
6      is necessarily a profile/seam leak into the pure core ? exactly what D1 forbids.
7
8      Three arms:
9      * L4a ? the WINDOW stage (`trajectory_to_action_windows`) is a pure, profile-
agnostic
10         staticmethod: deterministic across calls + its signature carries no
profile/compute/storage
11         parameter (so it structurally cannot branch on a profile).
12         * L4b ? the real stub-backed segmenter produces byte-identical segment ranges
under

```

```

13         `profile=truly-local` vs `profile=cpu-mock` vs no profile (the only varied knob is
the
14         compute target, which the core never reads).
15         * **L4c** ? the worker (`cmd_infer`, the cloud-serving orchestration) writes an
`intervals.csv`
16         whose ranges match the in-process truly-local run for the same stub trajectory.
17         ""
18
19     import inspect
20     import json
21
22     from backend.config import load_config
23     from backend.infrastructure.database import Database
24     from backend.pipeline.detectors.stub_runner import StubDetectorRunner
25     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
26
27     # A fixed duration so process_video's early ffprobe check passes for the fake
(undecodable) video;
28     # applied identically to every arm so it can never be the source of a divergence.
29     _DURATION = 30.0
30
31
32     def _config(tmp_path, profile):
33         """A config.json applying `profile` (or none), forced to the deterministic CPU stub
+
34         fully-local (no AI handoff) + a tmp output dir. (The stub uses its default synth
params ?
35         `indexing.stub` is not a typed knob, so it deterministically self-configures.)"""
36         data = {
37             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
38             "output": {"output_dir": str(tmp_path / "out")},
39         }
40         if profile:
41             data["deployment"] = {"profile": profile}
42             cfg_file = tmp_path / f"config_{profile or 'none'}.json"
43             cfg_file.write_text(json.dumps(data))
44             return load_config(cfg_file)
45
46
47     def _ranges(segments):
48         """Normalise a segment list to comparable (start, end) tuples (rounded to drop float
noise)."""
49         out = []
50         for s in segments:
51             start = s.get("start_time", s.get("start"))
52             end = s.get("end_time", s.get("end"))
53             out.append((round(float(start), 3), round(float(end), 3)))
54         return out
55
56
57     def _inprocess_ranges(tmp_path, profile, monkeypatch):
58         tmp_path.mkdir(parents=True, exist_ok=True)
59         # The fake video is undecodable; pin the duration probe so the early validity check
passes.
60         # Applied to the shared module fn ? identical for every profile arm (never a
divergence source).
61         monkeypatch.setattr(
62             "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
63             lambda p: _DURATION,
64         )
65         db = Database(str(tmp_path / f"db_{profile or 'none'}.sqlite"))
66         cfg = _config(tmp_path, profile)
67         video = tmp_path / "in.mp4"
68         video.write_bytes(b"FAKEVIDEO-PARITY")
69         db.add_video(

```

```

70         "vidParity", str(video), "processed", []
71     ) # FK target for candidate/play rows
72     seg = WasbHybridSegmenter(db, cfg)
73     result = seg.process_video("vidParity", str(video))
74     return _ranges(result)
75
76
77     # ----- L4a (pure WINDOW
stage)
78
79
80     def test_window_stage_has_no_profile_parameter():
81         """Structural guard: the WINDOW stage (`trajectory_to_action_windows`) carries no
82         profile/compute/storage parameter, so it *cannot* branch on a deployment profile ?
the
83         precise leak D1 forbids. (Determinism across profiles is exercised end-to-end in
L4b.)"""
84         sig = inspect.signature(StubDetectorRunner.trajectory_to_action_windows)
85         banned = {"profile", "compute_target", "deployment", "storage", "backend"}
86         assert not (banned & set(sig.parameters)), (
87             f"WINDOW stage must not take a profile/compute/storage param; got
{list(sig.parameters)}"
88         )
89
90
91     # ----- L4b (in-process
parity)
92
93
94     def test_inprocess_segment_ranges_identical_across_profiles(tmp_path, monkeypatch):
95         """Same stub trajectory + config under different profiles ? byte-identical segment
ranges.
96         The profiles differ only in compute_target/label, which the pure core never
reads."""
97         local = _inprocess_ranges(tmp_path / "a", "truly-local", monkeypatch)
98         mock = _inprocess_ranges(tmp_path / "b", "cpu-mock", monkeypatch)
99         none = _inprocess_ranges(tmp_path / "c", "", monkeypatch)
100         assert local == mock == none
101         assert (
102             len(local) >= 1
103         ) # the stub actually produced rallies (the test isn't vacuous)
104
105
106     # ----- L4c (worker
orchestration)
107
108
109     def test_worker_intervals_match_inprocess(tmp_path, monkeypatch):
110         """The worker (cmd_infer ? the cloud-serving orchestration) serialises an
intervals.csv whose
111         ranges equal the in-process truly-local run for the same stub trajectory. Proves the
112         fetch/store/serialise orchestration doesn't perturb the core."""
113         from backend.worker import __main__ as wmain
114
115         # In-process reference ranges (this also pins get_video_duration for the worker run
below).
116         ref = _inprocess_ranges(tmp_path / "ref", "truly-local", monkeypatch)
117
118         # Worker path: stub + skip-AI, validators mocked (fake bytes aren't a real video).
119         class _OK:
120             passed = True
121             validator_name = "fake"
122             message = "ok"
123
124             def model_dump(self):

```

```

125         return {"validator_name": "fake", "passed": True, "message": "ok"}
126
127     monkeypatch.setattr(
128         wmain.validator_registry,
129         "run_all_with_context",
130         lambda path, cfg: ([_OK()], {}),
131     )
132     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
133
134     cfg_file = tmp_path / "wcfg.json"
135     cfg_file.write_text(
136         json.dumps(
137             {
138                 "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
139                 "output": {"output_dir": str(tmp_path / "wout")},
140             }
141         )
142     )
143     video = tmp_path / "w.mp4"
144     video.write_bytes(b"FAKEVIDEO-PARITY")
145     out = tmp_path / "bucket"
146
147     rc = wmain.cmd_infer(
148         wmain.build_parser().parse_args(
149             [
150                 "infer",
151                 "--video-uri",
152                 str(video),
153                 "--out-uri",
154                 str(out),
155                 "--config",
156                 str(cfg_file),
157                 "--scratch",
158                 str(tmp_path / "scratch"),
159                 "--segmenter",
160                 "wasb_hybrid",
161             ]
162         )
163     )
164     assert rc == 0
165
166     # Find the intervals.csv (outputs/<video_id>/intervals.csv).
167     intervals = next(out.rglob("intervals.csv"))
168     rows = intervals.read_text().splitlines()[1:] # drop header
169     worker = [
170         (round(float(r.split(",")[0]), 3), round(float(r.split(",")[1]), 3))
171         for r in rows
172     ]
173     assert ref # non-vacuous: the stub produced rallies (so this isn't [] == [])
174     assert worker == ref
175

```

16. assistant (2026-07-08T06:28:11.552Z)

Let me read the full `\_execute\_processing` function to understand config resolution and the shim.

17. user (2026-07-08T06:28:12.286Z)

```

720
721
722     def _execute_processing(
723         config=None, db=None, req=None, progress=None, trusted_video_id=None
724     ) -> Dict[str, Any]:
725         """The full hash ? validate ? segment ? report pipeline for one video.

```

```

726
727     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
728     module-level ``global_config``/``db_instance`` (backward-compatible with existing
729     tests that pass only ``req``). Route handlers always pass explicit params; older
730     test code that calls ``_execute_processing(req)`` still works.
731
732     ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
733
734     ``trusted_video_id`` is the SERVER-DERIVED submit-time md5 the drain already knows
for this
735     job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ? byte-identical to
736     ``compute_video_id`` and already trusted for the submit-time cache probe. When
present (and
737     ``deployment.trust_submit_video_id`` is on), it is reused so the drain SKIPS re-
hashing the
738     multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes
as before.
739     It is a keyword-only, internal channel ? NOT a client-supplied request field (a
caller's
740     ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative
id).
741
742     Raises on unexpected internal errors ? the caller records those as a job failure
743     ""
744     import backend.main as _main # call-time seam: main's globals/model stay patchable
745
746     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
747     # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
748     if config is not None and not isinstance(config, AppConfig):
749         # Old signature: _execute_processing(req, progress=None)
750         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
751             req = db
752             db = None
753         elif req is None and isinstance(config, _main.ProcessRequest):
754             req = config
755             config = None
756     # Resolve config/db from module globals when not passed explicitly
757     if config is None:
758         config = _main.global_config
759     if db is None:
760         db = _main.db_instance
761
762     notify = progress or (lambda stage: None)
763
764     notify("hashing")
765     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
766     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
767     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
768     # consumers (hash / validators / segmenter) use the resolved local path.
769     local_video = ""
770     try:
771         local_video = storage.acquire_local_input(
772             req.video_path, getattr(config, "storage", None)
773         )
774         # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing
the whole
775         # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
776         # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the
submit-time
777         # cache probe trusts, so reusing it here is consistent, not a new trust

```

```

assumption. Gated by
778         # deployment.trust_submit_video_id (default on) and only honoured when the id is
well-formed
779         # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The
fetch above
780         # is unchanged; only the redundant hash is skipped.
781         trust_submitted = bool(
782             getattr(
783                 getattr(config, "deployment", None), "trust_submit_video_id", True
784             )
785         )
786         # STRICT: trust only an id in the EXACT form the drain would have produced
(32-char
787         # lowercase hex, per compute_video_id / submit_time_video_id). No normalization
? a
788         # differently-cased or malformed id is a signal something is off, so re-hash
rather than
789         # silently coerce it into the DB key.
790         candidate = trusted_video_id or ""
791         if trust_submitted and _looks_like_video_id(candidate):
792             video_id = candidate
793             logger.info(
794                 "[API] trusting submit-time video_id=%s ? skipped the drain re-hash
(#524).",
795                 video_id,
796             )
797         else:
798             video_id = _main.compute_video_id(local_video)
799     except Exception:
800         storage.cleanup_acquired_local_input(
801             req.video_path, local_video, getattr(config, "storage", None)
802         )
803         raise
804     try:
805         existing_video = db.get_video(video_id)
806     except Exception:
807         storage.cleanup_acquired_local_input(
808             req.video_path, local_video, getattr(config, "storage", None)
809         )
810         raise
811     was_reingest = existing_video is not None
812
813     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
814     default_seg = getattr(
815         getattr(config, "indexing", None), "default_segmenter", "motion"
816     )
817     seg_name = req.segmenter or default_seg
818     segmenter_actual = None
819     segmentation_ran = False
820     val_results: List[Any] = []
821     val_context: Dict[str, Any] = {}
822     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
823     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
824     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
825     compute_actual: Optional[str] = None
826     response: Optional[Dict[str, Any]] = None
827     try:
828         if (
829             existing_video
830             and existing_video.get("status") == "processed"
831             and not req.force
832         ):

```

```

833         cached_segments = db.query_play_segments(video_id, {})
834     if cached_segments:
835         logger.info(
836             "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
837             video_id,
838         )
839     response = _cached_process_payload(
840         video_id,
841         existing_video.get("validation_results", []),
842         cached_segments,
843         getattr(req, "compute", None),
844     )
845     return response
846
847     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
848     #     UNLESS this exact content already has a cached verdict under the current
validator config.
849     notify("validating")

```

18. assistant (2026-07-08T06:28:29.936Z)

Let me verify the production wiring ? that `jobs.video\_id` is populated with
`submit\_time\_video\_id` at submit time (so win #2 actually fires), and read the
`\_run\_claimed\_job` context.

19. user (2026-07-08T06:28:32.753Z)

```

150     the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151     result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
152     job self-scheduled and will be re-claimed when `next_poll_at` is due.
153     """
154     import backend.main as _main
155
156     job_id = job["id"]
157     # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158     # original /api/process pipeline). Both record their terminal state the same way;
only process
159     # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160     is_compile = job.get("kind") == "compile"
161     try:
162         if is_compile:
163             result = _main._execute_compile(
164                 config,
165                 db,
166                 job,
167                 progress=lambda stage: db.set_job_progress(job_id, stage),
168             )
169         else:
170             req = _main._job_to_request(job)
171             result = _main._execute_processing(
172                 config,
173                 db,
174                 req,
175                 progress=lambda stage: db.set_job_progress(job_id, stage),
176                 # #524 win #2: hand the drain the submit-time md5 already on the job row
177                 # ( `submit_time_video_id` , #484 ? server-derived, byte-identical to
178                 # compute_video_id) so it can skip re-hashing the fetched clip. "" ?
unknown
179                 # (local upload / non-gcs URI) ? the drain hashes as before.
180                 trusted_video_id=job.get("video_id") or None,

```



```

181         )
182         if result.get("video_id"):
183             db.set_job_video_id(job_id, result["video_id"])
184         db.mark_job_done(job_id, result)
185         if not is_compile:
186             _maybe_record_job_cost(
187                 job, result, config, db
188             ) # M8 cost ledger (default-OFF; process jobs only)
189             _maybe_run_flywheel(
190                 job, result, config
191             ) # M11 data-flywheel (process jobs only)
192     except _main.JobWaiting as w:
193         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
194         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")

```

20. user (2026-07-08T06:28:32.776Z)

```

30:     db: Any, job_id: str, req: Any, owner_id: Optional[str], video_id: str = ""
38:     hashed until the worker runs, so the submitted ref is the stable pre-flight key.
`video_id` is
51:         video_id=video_id,
92:         per video_id) and completes the job instantly: no drain, no GPU dispatch.
98:     video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
99:     probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
100:         db, cfg, video_id
104:     outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
113:         db, cfg, video_id, req.video_path, requested_compute=req.compute
185:         "video_id": j.get("video_id"),

```

21. assistant (2026-07-08T06:28:49.511Z)

Let me check the README diff and whether DOC_STATUS / docs/README enumerate docs that need registering.

22. user (2026-07-08T06:28:52.508Z)

```

diff --git a/docs/COMPUTE_DECOUPLED_SERVING/README.md b/docs/COMPUTE_DECOUPLED_SERVING/README.md
index e60d644..d0f3fea 100644
--- a/docs/COMPUTE_DECOUPLED_SERVING/README.md
+++ b/docs/COMPUTE_DECOUPLED_SERVING/README.md
@@ -11,6 +11,8 @@

```

> **Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive**, next to the source. Two design promises around it: a user can **switch local?cloud anytime** (D13, a headline feature), and execution is **never a surprise** ? auto-detect **suggests**, the user **confirms** (D15).

+> **#524 (2026-07-08):** the **L4-primary topology** question ? "collapse upload?stitch onto the worker, shrink the control-plane to a thin router" ? is resolved in [L4_PRIMARY_TOPOLOGY.md](./L4_PRIMARY_TOPOLOGY.md). It **recommends against** uploading directly onto the GPU box (a cost model shows that trades a ~free intra-region LAN pull for minutes of idle GPU) and instead **collapses validate+detect+stitch** onto the scale-to-zero L4 Job via #521's **phase_placement** ? honoring D4/D7/D16. Ships **win #2** here (drop the redundant control-plane re-hash; trust the submit-time md5, reversible).

+
0. TL;DR
The rally engine is a **pure function** of (input bytes + config) ? outputs in three deterministic stages ? **DETECT** ? **WINDOW** ? **STITCH** ? that **no deployment profile** may branch on. A **ComputeBackend** (deployment profile) decides **where** the core runs (a local process vs a Cloud Run GPU job); a **StorageBackend** decides **where bytes** come from (local FS / USB / mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local** (local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload <2GB / gdrive / bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus

****cpu-mock**** for CI. The single most important caveat: detection is ****byte-identical** only on the same GPU¹ (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the ****parity guarantee** is enforced at the WINDOW + STITCH layer², not detector-CSV bytes.

23. user (2026-07-08T06:28:54.187Z)

```
architecture.svg
L4_PRIMARY_TOPOLOGY.md
README.md
runlogs
TESTING_STRATEGY.md
--- DOC_STATUS register check ---
10:- **Reviewing now? Read §4 (the open findings/actions) ? that's what needs attention. §3 is the full register.
12:- **This register does NOT duplicate completeness detail ? it indexes lifecycle + archive-readiness and points to each tracked doc's §7.
28:## 3. The register
34:| `COMPUTE_DECOUPLED_SERVING/` | **IN PROGRESS ? M0 approved + **M1?M9 LARGELY SHIPPED (#279?#290; M5 #286, M6 #287, M8 #288, M9-auth #290 all merged, default-OFF); Gen-0 weights pushed (`weights/0.1.0-l4/`) | No | M11 flywheel + M12 gdrive-api (owner residual: counsel/prices/OAuth/real-run within the D17 cap) |
60:| `PLATFORM_ARCHITECTURE.md` | Strategy/research | The `ComputeBackend`/`StorageBackend` registries ? realized by COMPUTE_DECOUPLED_SERVING |
85:**ADR-001 ? ADR-014 (14, all ratified). The compute/serving lineage: **ADR-005 ? 008 ? 009 ? 010 ? 011 ? 012 ? 013 ? 014. ADR-014 was **RATIFIED 2026-06-21 (owner M0 sign-off for COMPUTE_DECOUPLED_SERVING).
95:6. **? RESOLVED (2026-06-21) ? `COMPUTE_DECOUPLED_SERVING` M0 approved + M1?M4 SHIPPED (#279/#280/#282/#283); all §8 gating Qs locked (D7?D17); M5+ default-OFF code batch authorized (owner residual: counsel ToS/DPA, real GCP prices, OAuth app, real cloud run within the D17 $100/?10k cap).
98:Update a row when its doc changes lifecycle; run §2 before any archive; re-review §4 at each clean-slate. This register + the §2 checklist are the operational form of the `CLAUDE.md` archive policy and [[working-agreement-tracked-project-docs]].
```

24. assistant (2026-07-08T06:29:00.641Z)

Let me read the new design doc to check it follows the PROJECT_DOC_TEMPLATE convention (status header, decisions-locked, progress tracker, DoD).

25. user (2026-07-08T06:29:01.745Z)

```
1      # L4-primary pipeline topology ? collapse the pipeline onto the worker, shrink the control-plane to a thin router
2
3      > **Status: `DRAFT` ? **owner-review (recommends *against* the epic's proposed direction #1; see §0/§8) . **Owner: Avi . **Created: 2026-07-08 . **Last updated: 2026-07-08
4      > **Lifecycle: `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
5      > **Issue: [#524](https://github.com/Khelsutra/badminton-highlight-indexer/issues/524) (epic) . **Builds on: [#521](https://github.com/Khelsutra/badminton-highlight-indexer/issues/521) (`phase_placement`) . **Motivated by: [#520](https://github.com/Khelsutra/badminton-highlight-indexer/issues/520) (the tracing that surfaced the data-movement finding)
6      > **Relation to existing docs: extends [COMPUTE_DECOUPLED_SERVING/README.md](./README.md) (ADR-014) and [PHASE_PLACEMENT.md](#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md); it does **not supersede them ? it answers the *topology* question they left open and sequences the migration on top of `phase_placement`.
7      > **Honesty note: each claim is tagged `[verified]` (checked against code/runlog), `[researched]` (needs sign-off / external pricing I could not fetch live), or `[design]` (proposed). Not legal/financial advice.
8
```

```

9      ---
10
11      ## 0. TL;DR
12
13      The 2026-07-08 live CUJ moved one 1.83 GiB clip **3** (upload?bucket, bucket?control-
plane to validate, bucket?worker to detect) and pinned the CPU-only 2-vCPU control-plane for
**~16 min** of heavy work (validation 183 s + stitch ~12m55s). The epic proposes fixing this by
**uploading directly onto the L4** and async-copying to the bucket. **This design recommends
*against* that direction** and quantifies why: a same-region bucket?compute transfer inside GCP
is **LAN-fast and not billed as internet egress** `[researched]`, so the round-trip that upload-
onto-L4 saves is worth **~seconds and ~$0**, while keeping an L4 up to **receive* a **minutes-
long** upload burns **~175 s of idle GPU per run (~$0.08?0.16)** `[design]` and **opens* a
durability exposure window instead of closing one.
14
15      The cheaper, more durable, already-ratified path is the opposite: **keep the upload
streaming to the cheap bucket** (the #480 design ? the durable copy exists **before* the L4 ever
starts), and **collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** ?
which is exactly what **#521's `phase_placement` already wires as a config flip**. The residual
**~30 s pre-validation gap** is then killed not by moving the **upload* but by the control-plane
**never downloading the file at all** (it delegates validation to the L4 per #521, and trusts
the already-known submit-time md5 per this epic's win #2 ? no re-hash). Net data movement drops
from 3x to the theoretical floor: **1 client upload + 1 LAN-fast intra-region pull**, with
**zero idle-GPU cost** and **zero exposure window**.
16
17      **Actionable now (this PR):** the design doc + **win #2** (drop the redundant control-
plane re-hash, config-guarded, reversible). **Win #1** (validation-on-L4) is **delivered by
#521** ? this doc does not duplicate its knob. A persistent GPU (GCE VM / Cloud Run Service-
with-GPU) is **rejected at current volume** on a quantified cost basis and left as an owner-
gated future ADR.
18
19      ## 1. Problem & goal
20
21      **Why now.** Tracing the live CUJ (`runlogs/DEPLOY_REPORT_cloud-serving_validation-
latency_2026-07-07.md` §9 `[verified]`) produced two findings:
22
23      1. **Data is moved 3x per run** `[verified]`: (a) upload streams straight to
`gs://?/uploads/?` (never touches control-plane disk ? the #480/#471 OOM fix,
`backend/api/uploads.py`), computing the md5 incrementally as it streams; (b) the control-plane
re-downloads the whole clip to scratch to validate ? `_execute_processing` ?
`storage.acquire_local_input(gs://?)` at `backend/orchestration.py:748`, then a **redundant re-
hash** `compute_video_id` at `:751`; (c) the worker downloads it a third time to detect
(`[worker]` pulled ?). The unlogged **~30 s pre-validation gap** is finding (b).
24
25      2. **The control-plane is the scaling wall** `[verified]`: validation (183 s) **and**
stitch (~12m55s, ~67 s/clip 4K re-encode, no NVENC) both run in-process on the 2-vCPU control-
plane, serialized by `_LOCAL_COMPUTE_LANE` ? ~16 min of heavy CPU per run ? the API saturates
past ~5 concurrent users.
26
27      **The concrete outcome we want.** The L4 worker becomes the primary compute for the
whole pipeline; the control-plane shrinks to a **thin router** (auth · dispatch · DB · SPA + job
status · signed-URL download) that does **no heavy decode/encode** and, ideally, **never pulls
the full file**. Every move must be **reversible by config** with a **bucket-mediated
fallback**, and land as small PRs on top of `origin/master`.
28
29      **The hard question this doc must resolve (compute topology).** "Upload directly to the
L4" needs an **inbound endpoint on the GPU box**, but the current L4 is a Cloud Run **Job**
(batch, no inbound HTTP ? `backend/compute/cloud_run.py`, dispatched per-request, `[verified]`).
So the epic forces a choice between **A. Cloud Run Service-with-GPU**, **B. GCE L4 VM**, and
**C. a hybrid** presign/redirect. §4 answers it with numbers.
30
31      **Read to get current:** `[PHASE_PLACEMENT.md`
(`#521`)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-
on-l4/docs/COMPUTE\_DECOUPLED\_SERVING/PHASE\_PLACEMENT.md) ? the `phase_placement` knob this
sequences on, `[README.md]`(`./README.md`) §2 decisions **D4/D7/D16** (the ratified constraints that
bound the options), and the runlog §9 (the evidence).

```

```

32      ## 2. Decisions locked
33
34      | # | Decision | Source / date | Implication |
35      |---|-----|-----|-----|
36      | L1 | **Keep upload ? bucket** (streaming, #480). The durable copy exists **before**
any compute runs; the bucket *is* the "reliable co-located disk" the epic wants ? no VM needed
for durability. | this doc `[design]` + #480 `[verified]` | No exposure window; upload cost
stays on the cheap I/O path. |
37      | L2 | **Collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job**
via `phase_placement` (#521), not onto a persistent GPU. Honors **D7** (scale-to-zero) +
**D16(a)** (Jobs, not Services). | #521 `[verified]` + D7/D16 | The "L4-primary" vision is
realized by **config flips**, not a rearchitecture. |
38      | L3 | **Kill the ~30 s gap by *not downloading to the control-plane*, not by uploading
onto the L4.** Compose #521's validation-delegation (CP skips the validator suite) with win #2
(CP trusts the submit-time md5 ? no re-hash). | this doc `[design]` | The CP stops pulling the
full file when validation is delegated; movement ? 3x2x. |
39      | L4 | **Reject persistent-GPU (GCE VM / Service-with-GPU) at current volume.**
Breakeven vs scale-to-zero is a **~51 % on-demand / ~31 % spot duty cycle** (? 2,600?4,300
runs/mo sustained) `[design]`; we are at pilot volume. Revisit only behind a **new ADR** at
sustained load. | $4.2 cost model `[design]` | Preserves per-run billing (D7/D8); avoids
~$382?637/mo idle. |
40      | L5 | **Reject "upload onto the L4."** It saves a ~free, seconds-fast intra-region LAN
pull at the cost of ~175 s idle GPU/run **and** opens a durability window. | $4.3 `[design]` |
The epic's proposed direction #1 is not adopted; owner sign-off requested ($8). |
41      | L6 | **Win #2 (drop the re-hash) is config-guarded + reversible**
(`deployment.trust_submit_video_id`, default on) and produces a **byte-identical `video_id`**
(the GCS `md5Hash` over the same bytes = `compute_video_id`). | win #2 `[verified]` | Zero-
regression; one-flag rollback. |
42
43      *(IDs are `L#` to avoid colliding with README.md's `D#`. These refine ? never override ?
D4/D7/D16.)*
44
45      ## 3. Foundation ? the ratified constraints this must live inside
46
47      The topology is **not a blank slate** ? ADR-014 (README.md §2) already ratified the
serving substrate. The options in §4 are scored **against** these, and any option that breaks
one needs a new ADR, not just this doc:
48
49      | Ratified decision | What it pins | Consequence for #524 |
50      |---|---|---|
51      | **D4** | In cloud-serving, **stitch runs on Cloud Run** (co-located, metered). | The
live system's CP-stitch is a *deviation* from D4; #521 *realizes* D4. #524 is mostly
**completing an already-ratified decision**, not inventing one. |
52      | **D7** | **Scale-to-zero, no warm pool** (a ~$500/mo idle GPU "would break per-video
billing"). | Any persistent-GPU option (B, Service-with-min-instances) **contradicts D7** ? new-
ADR territory (§4.2). |
53      | **D16(a)** | **Cloud Run Jobs** (not Services-with-GPU), **bucket-poll** for
completion. | Option A (Service-with-GPU inbound upload) **contradicts D16(a)**. The Job model
is load-bearing (reuses `JobWaiting`/`claim_due_job`). |
54      | **D16(b)** | Stitch encode = libx264 for parity; **NVENC deferred behind a knob**. |
#521 lands exactly that knob (`cloud_stitch_encoder`), using this CUJ as the cost data D16(b)
said to revisit on. |
55      | **D16(c)** | Input cap hard-reject `< 2 GB`. | The 1.83 GiB CUJ clip is *at* the cap;
the topology must stay sane at 2 GB (upload minutes, pull seconds). |
56
57      **Key reframing this foundation gives us:** the owner's "reliable persistent disk"
requirement is *already met* by the **regional GCS bucket** ? a durable, co-located, cheap
"disk" the upload already writes to directly. The persistent-VM instinct solves a problem
(durable local disk) that the bucket-first design doesn't have.
58
59      ## 4. Design / architecture ? resolving the topology question
60
61      ### 4.1 The measured pipeline (evidence base) `[verified]`
62
63      From runlog §9 (rev `rally-control-plane-00019`, "First CUJ.MP4", 1.83 GiB / 174.17 s /

```

```

4K-class):
64
65     | Phase | Wall | Machine | GPU needed? | Notes |
66     |---|---|---|---|---|
67     | Upload (client ? bucket) | **~2m55s** | client uplink | no | 1.83 GiB @ ~84 Mbps ?
**owner bandwidth**, not infra |
68     | Accept ? validate start | ~30 s | control-plane | no | **the gap** = CP re-download to
scratch (`acquire_local_input` + re-hash) |
69     | Validation (#512 one-pass) | **183 s** | control-plane 2 vCPU | no (CPU-bound) |
scene_cut/blur/relevance shared decode |
70     | Dispatch ? worker ready | ~55 s | Cloud Run Job | ? | image pull + GCSFuse +
torch/cuda init (cold start) |
71     | Detection (WASB) | **161.6 s** | L4 (cuda) | **yes** | the only genuinely GPU-bound
phase |
72     | Ingest + finalize | ~4 s | control-plane | no | intervals ? DB |
73     | Compile / stitch | **~12m55s** | control-plane 2 vCPU | no (NVENC would need GPU) |
11x 4K re-encode+watermark |
74
75     **Two facts that drive the whole design:**
76     - **Only *detection* is GPU-bound.** Validation is CPU/decode-bound; stitch is encode-
bound (NVENC *wants* a GPU but libx264 doesn't *need* one). So "collapse onto the L4" is about
**co-locating CPU work next to the GPU on a box that's already up and already holding the
bytes**, not about the GPU per se.
77     - **Upload dwarfs compute** (175 s upload vs 161.6 s GPU-detect). Anything that keeps a
GPU allocated *during upload* roughly **doubles** the GPU-allocated time for zero compute.
78
79     ### 4.2 Cost model (asia-southeast1, USD, Cloud Billing Catalog API, 2026-07-08)
`[verified]` rates / `[design]` per-run
80
81     Rates pulled live from the Catalog API (`cloudbilling.googleapis.com`, project `gen-
lang-client-0306026826`):
82
83     | Resource | Rate | Source |
84     |---|---|---|
85     | Cloud Run **L4 GPU** (no zonal redundancy) | **$0.00022404 / s** = $0.806/hr | Catalog
`[verified]` |
86     | Cloud Run vCPU (instance/jobs) | $0.0000216 / s | Catalog `[verified]` |
87     | Cloud Run memory | $0.0000024 / GiB·s | Catalog `[verified]` |
88     | ? **L4 Job @ 8 vCPU / 32 GiB (running)** | **$1.705 / hr** ($0.000474/s) | derived
`[design]` |
89     | GCE **g2-standard-4** (4 vCPU/16 GiB + L4), on-demand | **$0.872 / hr** = **~$637 /
mo** | Catalog `[verified]` |
90     | GCE g2-standard-4, **spot/preemptible** | $0.523 / hr = ~$382 / mo | Catalog
`[verified]` |
91     | Cloud Run **GPU Service, warm** (min-instances=1, 4 vCPU/16 GiB) | ? $1.256 / hr =
**~$917 / mo** idle | derived `[design]` |
92     | Balanced PD | $0.11 / GiB·mo | Catalog `[verified]` |
93
94     **Per-run GPU-allocated cost (the term that varies by topology):**
95
96     | Topology | GPU-allocated time / run | GPU cost / run | ? vs today | Concurrency wall?
|
97     |---|---|---|---|---|
98     | **Today** (detect-only on L4; validate+stitch on CP) | ~222 s (cold+detect+io) |
**~$0.105** | ? | **Yes** ? ~16 min CP CPU/run |
99     | **#521 + win #2** (validate+detect+stitch on L4; upload?bucket) | ~312 s (validate 60
+ detect 162 + stitch 60 + io 30) | **~$0.148** | **+$0.04** | **No** ? CP is thin |
100    | **Upload-onto-L4** (GPU up during upload too) | ~487 s (+175 s idle upload) |
**~$0.231** | **+$0.13** | No, but idle-GPU tax |
101    | **Persistent GCE VM** (on-demand) | billed 24/7 regardless | **$637/mo flat** | breaks
per-run billing | No |
102
103    **Breakeven (persistent vs scale-to-zero):** a persistent on-demand g2 ($0.872/hr) beats
the scale-to-zero Job ($1.705/hr *while running*) only at a **duty cycle ? $0.872/$1.705 ? 51
%** (spot: ? 31 %). At ~312 s/run that is **~2,600?4,300 runs/month sustained, 24/7**

```

`[design]`. Pilot volume is **handfuls/day**. ***Scale-to-zero wins by ~2 orders of magnitude*** and preserves per-run billing (D7/D8). A persistent GPU is also ***idle >95 % of the time*** at our volume ? the ~\$382?637/mo would be almost pure waste.

104

105 ### 4.3 The three topology options, scored

106

107 ***Option A ? Cloud Run Service-with-GPU hosting upload + pipeline.***

108 - **Latency:* scale-to-zero ? same ~55 s cold start as the Job; ***min-instances=1***

removes cold start but pins a ***~\$917/mo warm idle GPU***.

109 - **Cost:* to accept the upload, the GPU Service instance must be ***up for the whole 175*

s upload** ? the idle-GPU tax of \$4.2 on **every** request, plus the warm-idle bill if min-

instances>0.

110 - **Ops/parity:* ***contradicts D16(a)*** (Jobs, not Services) and ***D7*** (scale-to-zero) ?

abandons the ``JobWaiting`/`claim_due_job`/bucket-poll` machinery that's shipped and tested. A

Service also needs request-timeout/concurrency tuning the batch Job sidesteps.

111 - ***Verdict: reject.*** Buys an inbound endpoint we don't need (the bucket is the inbound endpoint) at the cost of the idle-GPU tax + two ratified-decision reversals.

112

113 ***Option B ? GCE L4 VM (persistent, reliable PD).***

114 - **Reliability:* real, attached, reliable PD ? the owner's stated appeal. But ***the*
bucket already gives us durable co-located storage** (L1), so the PD solves a non-problem.

115 - **Cost:* ***\$382?637/mo flat***, >95 % idle at pilot volume; breaks per-run billing (D7/D8). Spot is cheaper but ***preemptible*** ? directly against the "reliable" motivation.

116 - **Ops:* we now ***own a VM*** ? patching, disk-fill, crash-recovery, autoscaling-group if we ever need >1. The Job model has none of this.

117 - ***Verdict: reject at current volume; owner-gated future ADR*** if sustained load crosses the \$4.2 breakeven.

118

119 ***Option C ? Hybrid: control-plane presigns/redirects the upload to a just-spun-up L4, pipeline runs there, async copy to bucket.***

120 - **The fatal timing problem:* compute needs the ***whole*** file, so the L4 cannot start useful work until the upload finishes. Spinning the L4 up **to receive** the upload means the GPU is ***allocated and idle for the entire 175 s upload*** ? the \$4.2 idle tax, same as A. Upload and compute ***cannot overlap***.

121 - **Durability:* introduces an ***exposure window*** (upload-complete ? async-mirror-confirmed) where the only copy is on L4 local disk; a preemption/crash in that window ***loses the source*** unless the client re-uploads (see §4.5).

122 - ***Verdict: reject.*** It pays the idle-GPU tax **and** takes on a durability window, to save a transfer that is ~free and seconds-fast.

123

124 ***Option A? (recommended) ? co-located Jobs, bucket-mediated upload (the L1?L3 design).***

125 - Keep ***upload ? bucket*** (cheap, bounded-memory, durable-immediately). Keep the ***scale-to-zero L4 Job***. Move ***validate + stitch*** onto the Job via ``phase_placement`` (#521).

Make the CP ***stop downloading*** (delegate validation + trust the md5, win #2). The Job pulls the file ***once*** from the same-region bucket at LAN speed.

126 - **Data movement:* ***1 client upload + 1 intra-region LAN pull*** ? the floor. (Detection and validation/stitch all run in ***one*** Job execution, so it's one pull, not two.)

127 - **Cost:* +\$0.04/run GPU vs today, ***~16 min CP CPU/run*** (the actual scaling win). No idle tax, no warm bill, no VM.

128 - **Durability:* ***no exposure window*** ? the bucket copy exists before the Job starts.

129 - **Ops/parity:* ***honors D4/D7/D16***; reuses the shipped dispatch/poll path; every phase is a ***config flip with an in-process fallback*** (#521's ``_dispatch*_remote`` already logs-and-falls-back to in-process when the box isn't cloud-wired).

130

131 > ***The one-line intuition:*** the bucket **is** the co-located reliable disk; the GPU should attach only for the ~5 min of compute, pulling from it at LAN speed ? never sit idle for the ~3 min upload.

132

133 ### 4.4 The minimal control-plane surface (the "thin router")

134

135 After A?, the control-plane does ***only*** I/O-light, decode-free work ? trivially horizontally scalable (the one caveat is SQLite; a Postgres move is out of scope, noted in §6):

136

137 | Keep on the control-plane | Why it must stay | Code anchor ``[verified]`` |

```

138 |---|---|---|
139 | **Cloudflare-Access edge auth** (verify `Cf-Access` JWT) | D9 ? one identity system;
direct-hit spoofing guard | `backend/api/auth.py`, gated at `/api/process` |
140 | **Upload endpoint** (streams parts ? bucket) | Bounded-memory, no decode (#480); the
durable-copy producer | `backend/api/uploads.py` |
141 | **Dispatch / routing** (submit job, dispatch Cloud Run Job, bucket-poll) | Owns the
async job lifecycle (`JobWaiting`/`claim_due_job`) | `backend/orchestration.py`,
`backend/compute/cloud_run.py` |
142 | **DB writes** (jobs, videos, `play_segments`, validation verdict) | The control-plane
owns the SQLite DB (worker uses a throwaway scratch DB) | `backend/infrastructure/database*.py`
|
143 | **SPA serving + job status** (`/api/jobs/{id}` polling) | The UX surface |
`backend/api/jobs.py`, `backend/ui/` |
144 | **Signed-URL download** (`/api/highlights` 307) | Reel is already in the bucket; CP
just signs | `backend/main.py::get_highlights` |
145 | **~~Validation~~** | ? **L4** via `phase_placement.validation` (#521) | delegated |
146 | **~~Stitch/compile~~** | ? **L4** via `phase_placement.compile` (#521) | delegated |
147 | **~~Full-file download + re-hash~~** | ? **eliminated** (delegate validation + trust
md5, win #2) | this PR |
148
149 ### 4.5 The async copy-to-bucket durability model (spec ? needed *only* if a future
upload-onto-L4 path is ever adopted)
150
151 The recommended A? design **has no async-copy step** (the bucket copy is the *upload*,
produced before compute). This section specs the model the epic asked for, so that if the owner
overrides S8 and adopts upload-onto-L4, the durability semantics are pinned rather than
improvised:
152
153 - **When the mirror runs:** kick a background `storage.put(local ? gs:///uploads/?)`
**immediately on upload-complete** (not "during segmentation" ? starting it later only lengthens
the exposure window; there's no compute reason to delay it). Run it concurrently with
validation/detect.
154 - **Failure/resume:** retry with capped exponential backoff; the mirror is
**idempotent** (same object key = the md5). On worker preemption before the mirror confirms, the
run is **lost** ? the job must fail with a *retryable* status so the client can re-submit (the
source is still on the client). **Never** report success until the mirror is **confirmed**
(object exists + size/md5 match).
155 - **Exposure window:** upload-complete ? mirror-confirmed, the **only** durable copy is
L4 local PD. Window length ? mirror wall (1.83 GiB intra-region ? seconds?tens of seconds
`[design]`). A preemption in-window = re-upload. **This window is the entire reason A? is
preferred** ? A?'s window is *zero*.
156 - **Ack contract:** the client keeps the source until the API returns
`mirror_confirmed:true`; only then may it release it. (A? needs no such contract ? bucket-
first.)
157
158 ## 5. Phased migration ? every step a config flip with a bucket-mediated fallback
159
160 Built on #521's `resolve_phase_placement(cfg, phase)` so **placement is config, not
code**. Each phase is independently reversible (`control_plane` ? `cloud_run_l4`) and falls back
to in-process on a non-cloud box.
161
162 | Phase | Change | Mechanism | Fallback | Status |
163 |---|---|---|---|---|
164 | **P0** | **Today** ? detect on L4; validate+stitch on CP | `compute_target=cloud_run` |
in-process | shipped [verified]` |
165 | **P1** | **Stitch ? L4** | `phase_placement.compile=cloud_run_l4` (+
`cloud_stitch_encoder`) | in-process stitch | **#521 (open)** |
166 | **P2a** | **Drop the CP re-hash** (trust submit-time md5) |
`deployment.trust_submit_video_id` (default on) | re-hash on the drain | **this PR (win #2)** |
167 | **P2b** | **Validation ? L4** (CP stops running validators) |
`phase_placement.validation=cloud_run_l4` (guards: requires cloud segment) | CP validates |
**#521 (open)** |
168 | **P2 net** | **CP never downloads the file** ? the ~30 s gap dies; movement 3x?2x |
P2a ? P2b together | either half alone degrades gracefully | needs #521 **** win #2 merged |
169 | **P3** | Worker serializes its **validation verdict** back (per-validator results +

```

#518 cache warm), so a worker-side reject renders the same SPA detail as a CP reject | new
`report.json`/marker field + CP ingest | worker reject = rc?0 + message (today) | **follow-up**
(see \$6) |

170 | **P4** | *If and only if* sustained load crosses the \$4.2 breakeven | persistent
worker (VM/Service) | scale-to-zero Job | **owner-gated, new ADR** |

171

172 **The "collapse onto the L4" the epic asks for = P1 + P2, both config flips on #521's
knob + this PR's win #2.** No rearchitecture, no new infra, fully reversible.

173

174 ## 6. Honest limits ? what this does NOT do / does NOT prove

175

176 - **A green suite does not prove the cloud path** ? the tests are fully-mocked/offline
(a fake `CloudRunJobClient`). P2's "CP never downloads" and the idle-GPU/round-trip numbers are
validated **in the cost model + code paths**, not on live infra. A live CUJ on rev ? `00020`
(owner-side, D17 budget) is required to *confirm* the gap is gone and the movement is 2x.

177 - **Network-egress "free intra-region" is `[researched]`, not live-verified** ? general
web egress is blocked here, so same-region GCS?Cloud Run being un-billed-as-internet-egress
rests on documented GCP behavior, not a fetched invoice. If it were *not* free, A? is *still*
cheapest (it moves the fewest bytes); the conclusion is robust to this uncertainty.

178 - **Win #2 changes nothing when no trusted id is present** ? local/appliance uploads and
non-GCS direct-URI submits (`submit\_time\_video\_id` ? `None`) still re-hash. The win applies to
the GCS cloud-serving path (the CUJ case).

179 - **P3 (worker validation-verdict serialization) is a real gap #521 leaves open**
`[verified]`: today a worker-side validation reject surfaces as `rc?0` + a message, but the per-
validator `ValidationResult` list (`result['results']` the SPA renders) and the #518
`validation\_cache` warm are **not** reconstructed on the CP. Until P3, moving validation to the
L4 (P2b) trades richer verdict UX + cache-warming for the CPU offload. Called out so P2b isn't
flipped on blind.

180 - **SQLite is the residual CP scaling limit** ? the thin router is stateless *except*
the SQLite DB; true horizontal scale-out needs Postgres (out of scope; noted).

181 - **The #513 asymmetry reverses under P2b** `[verified]`: worker-side validators run at
model-default thresholds unless the CP forwards `validation.\*` as `--set`. #521's validation-
delegation path must forward those (or re-inherit the divergence that caused the #513 incident).
Flagged for the #521 review, not owned here.

182

183 ## 7. Deliverables & progress tracker ? source of truth

184

185 Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, branched from
`origin/master`, no stacking.

186

187 | ID | Deliverable | Depends on | Gated? | Status | PR |

188 |----|-----|-----|-----|-----|----|

189 | P0 | **This design doc** (resolves topology A/B/C; recommends A?; specs durability +
phased migration) | ? | owner-review | ? (this PR) | #524 |

190 | P1 | **Win #2** ? drop the CP re-hash; trust submit-time md5;
`deployment.trust\_submit\_video\_id` (default on), reversible | ? | No | ? (this PR) | #524 |

191 | P2 | **Win #1** ? validation-on-L4 (`phase\_placement.validation`) | #521 | No | ?
delivered by #521 ? not duplicated here | #521 |

192 | P3 | **CP-download-avoidance measured live** ? confirm the ~30 s gap is gone +
movement is 2x on rev ? `00020` | #521 + P1 merged | ? owner-side (live spend, D17) | ? | ? |

193 | P4 | **Worker validation-verdict serialization** (\$6 gap) so P2b keeps rich SPA
verdict + #518 cache | #521 | No | ? follow-up | ? |

194 | P5 | **Persistent-GPU re-evaluation** only if load crosses \$4.2 breakeven | volume | ?
new ADR | ? | ? |

195

196 ## 8. Open questions ? owner / external

197

198 **Blocking (owner sign-off):**

199 1. **Do you accept rejecting "upload directly onto the L4" (L5)?** The epic proposed it;
this doc recommends against it with the \$4.2/\$4.3 idle-GPU-vs-free-LAN-pull numbers. A? delivers
the same "collapse onto the L4" outcome, cheaper and more durable, via config. *If you still
want upload-onto-L4* (e.g. for a future no-bucket edge deployment), \$4.5 specs its durability
model ? but it needs a Service-with-GPU or VM (a D16(a)/D7 reversal ? new ADR).

200 2. **Flip P2b (validation?L4) before or after P3 (verdict serialization)?** P2b alone
offloads CPU but degrades the reject-UX + skips #518 cache-warm (\$6). Recommend: land P3 first,


```

then P2b.
201
202     **Nice-to-know:**
203     3. Is pilot volume expected to cross the $4.2 breakeven within the planning horizon?
(Decides whether P5 is ever live.)
204
205     ## 9. Definition of done
206
207     - [x] Design doc resolves the topology question (A/B/C scored) with a clear
recommendation (A?) + quantified cost/latency/reliability. `[this PR]`
208     - [x] Minimal control-plane surface defined ($4.4). `[this PR]`
209     - [x] Async copy-to-bucket durability model specified with failure/resume + exposure
window ($4.5). `[this PR]`
210     - [x] Phased migration laid out on #521's `phase_placement`, each step a config flip
with a bucket-mediated fallback ($5). `[this PR]`
211     - [x] Win #2 landed (config-guarded, reversible, tests green). `[this PR]`
212     - [ ] Win #1 confirmed delivered by #521 (not duplicated). `[cross-PR]`
213     - [ ] Live CUJ confirms the gap is gone + movement is 2x (P3, owner-side). `[gated]`
214     - [ ] Owner signs off on L5 (reject upload-onto-L4) ? flip Status ? IN PROGRESS/DONE.
215
216     ## 10. References
217
218     **Internal code anchors `[verified]`:** `backend/api/uploads.py` (streamed md5),
`backend/orchestration.py:220-239` (`submit_time_video_id`, #484) / `:741-763` (re-hash) /
`_maybe_dispatch_remote`, `backend/api/jobs.py:80-153` (submit-time cache already trusts the
md5), `backend/api/job_worker.py:122-193` (`_job_to_request`/`_run_claimed_job`),
`backend/compute/cloud_run.py` (dispatch + bucket-poll), `backend/config/placement.py` (#521
`resolve_phase_placement`), `backend/worker/__main__.py` (worker validation + `--skip-
validation`).
219     **Internal docs:** `[PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE\_DECOUPLED\_SERVING/PHASE\_PLACEMENT.md), [README.md](./README.md) (ADR-014,
D4/D7/D16), `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9
(evidence).
220     **External `[verified]` rates:** GCP Cloud Billing Catalog API
(`cloudbilling.googleapis.com`, asia-southeast1, USD, 2026-07-08). **`[researched]`** same-
region GCS?Cloud Run egress treatment (not live-fetchable here).
221
222     ---
223
224     ### Changelog
225     - `2026-07-08` ? initial draft: topology A/B/C resolved (recommend A? = co-located Jobs
+ bucket upload; reject persistent-GPU + upload-onto-L4 with a cost model); durability + phased
migration on #521's `phase_placement`; win #2 landed alongside.
226

```

26. assistant (2026-07-08T06:29:49.826Z)

The design doc references code with line anchors. Let me verify those line anchors against the post-change file, check the #521 merge-conflict surface, presets parity, worktree cleanliness, and docs/README indexing.

27. user (2026-07-08T06:29:54.678Z)

```

=== worktree status ===
M backend/api/job_worker.py
M backend/config/models.py
M backend/orchestration.py
A docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md
M docs/COMPUTE_DECOUPLED_SERVING/README.md
M tests/test_config_deployment.py
A tests/test_video_id_trust.py
=== line 748/751/798 (post-change) ===
    if config is not None and not isinstance(config, AppConfig):
        req = db

```

```
local_video = storage.acquire_local_input(
    video_id = _main.compute_video_id(local_video)
```

28. user (2026-07-08T06:30:03.549Z)

```
=== phase_placement present? ===
=== presets/model shape test ===
tests/test_config_deployment.py:12:- The model's Literal enum stays in parity with
backend/config/presets.PROFILE_PRESETS.
tests/test_config_deployment.py:27:    PROFILE_PRESETS,
tests/test_config_deployment.py:87:    assert set(PROFILE_PRESETS) == set(PROFILES)
tests/test_config_deployment.py:425:    assert DeploymentConfig().remote_poll_max_wait_sec ==
3900.0
tests/test_config_deployment.py:427:    DeploymentConfig(remote_poll_max_wait_sec=0)
tests/test_config_deployment.py:429:    DeploymentConfig(remote_poll_max_wait_sec=-1)
tests/test_config_wasb.py:168:    from backend.config.presets import PROFILE_PRESETS
tests/test_config_wasb.py:171:    cfg = AppConfig(**{"deployment": {"profile": "cloud-serving"},
**PROFILE_PRESETS["cloud-serving"]})
tests/test_video_id_trust.py:119:    cfg =
AppConfig(deployment=DeploymentConfig(trust_submit_video_id=False))
=== docs/README index of COMPUTE_DECOUPLED ===
20:- [COMPUTE_DECOUPLED_SERVING/](COMPUTE_DECOUPLED_SERVING/README.md) ? **? Active tracked
project (2026-06-21, `DRAFT` owner-gated):** the productionization successor to
DECOUPLED_COMPUTE (its deferred M3/M4). **One pure core (DETECT?WINDOW?STITCH) that never
branches on deployment profile**, with a `ComputeBackend` (where it runs) + `StorageBackend`
(what bytes) selected by **one startup config**. Ships **truly-local** (local/USB + local GPU,
in-process stitch, zero cloud) + **cloud-serving** (upload<2GB / gdrive / bucket ? a **Cloud Run
GPU job runs detect AND stitch**, metered into a per-job cost ledger) + **cpu-mock** (CI).
Includes [TESTING_STRATEGY.md](COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md) (profile-PARITY
proof for $0), [architecture.svg](COMPUTE_DECOUPLED_SERVING/architecture.svg), a
[runlogs/](COMPUTE_DECOUPLED_SERVING/runlogs/) posterity dir, and **ADR-014** ([lineage
ADR-009?010?011?012?013?014](archives/decisions/DECISIONS.md)). $7 M0?M10 tracker is the source
of truth.
36:- [`data_pipeline/`](data_pipeline/README.md) ? **the isolated data pillar**: how footage is
recorded (`VIDEO_RECORDING_GUIDELINES`), golden sets are created + which videos
(`GOLDEN_SET_IMPLEMENTATION` · `GOLDEN_SET_PHASE1_VIDEOS` · `GOLDEN_VIDEOS`), and data is
shared/ingested (`GOLDEN_DATA_SHARING`). Deliberately **no segmentation/stitching/model** detail
? that's the ML pillar (top level + `COMPUTE_DECOUPLED_SERVING/`). The **annotation-vendor
(Roora)** engagement is archived under `archives/roora-vendor/`.
```

29. assistant (2026-07-08T06:30:39.304Z)

Let me confirm the two full-dict assertions are the only shape-pinning ones, and inspect the presets parity assertion context.

30. user (2026-07-08T06:30:40.885Z)

```
55     mutates detector_impl / skip_ai_handoff / storage.backend)."""
56     monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
57     cfg = load_config(NONEXISTENT)
58     assert cfg.indexing.detector_impl == "auto"
59     assert cfg.indexing.skip_ai_handoff is False
60     assert cfg.storage.backend == "local"
61
62
63     def test_default_config_dump_only_adds_deployment_section():
64         """Adding the section must not perturb any other default ? the dump equals a fresh
65         AppConfig()'s dump, and the new section is present with empty values."""
66         dumped = load_config(NONEXISTENT).model_dump(mode="json")
67         assert dumped == AppConfig().model_dump(mode="json")
68         assert dumped["deployment"] == {
69             "profile": "",
70             "compute_target": "",
71             "max_concurrent_remote_jobs": 4,
```

```

72         "remote_poll_max_wait_sec": 3900.0,
73         "trust_submit_video_id": True,
74     }
75
76
77     # -----
78     # Parity: the model Literal must match the presets table (no drift).
79     # -----
80     def test_profile_literal_matches_presets():
81         import typing
82
83         literal_profiles = set(
84             typing.get_args(DeploymentConfig.model_fields["profile"].annotation)
85         )
86         assert literal_profiles == {"", *PROFILES}
87         assert set(PROFILE_PRESETS) == set(PROFILES)
88         assert set(COMPUTE_TARGET_FOR_PROFILE) == set(PROFILES)
89
90
91     def test_compute_target_literal_matches_presets():
92         import typing
93
94         literal_targets = set(

```

31. assistant (2026-07-08T06:30:56.367Z)

Let me verify the new tests actually run and pass (offline), and check the ValidationResult path.

32. user (2026-07-08T06:31:27.629Z)

```

..... [100%]
42 passed in 16.29s

```

33. assistant (2026-07-08T06:32:09.584Z)

All 42 pass. Let me pin exact line numbers for the config field and the doc anchors for precise reporting.

34. user (2026-07-08T06:32:12.652Z)

```

493:     trust_submit_video_id: bool = True
496:class BillingConfig(BaseModel):
=== doc anchors ===
23:1. Data is moved 3x per run [verified]: (a) upload streams straight to
`gs://?/uploads/?` (never touches control-plane disk ? the #480/#471 OOM fix,
`backend/api/uploads.py`), computing the md5 incrementally as it streams; (b) the control-plane
re-downloads the whole clip to scratch to validate ? `_execute_processing` ?
`storage.acquire_local_input(gs://?)` at `backend/orchestration.py:748`, then a redundant re-
hash `compute_video_id` at `:751`; (c) the worker downloads it a third time to detect
(`[worker]` pulled ?). The unlogged ~30 s pre-validation gap is finding (b).
218:Internal code anchors [verified]: backend/api/uploads.py (streamed md5),
`backend/orchestration.py:220-239` (`submit_time_video_id`, #484) / `:741-763` (re-hash) /
`_maybe_dispatch_remote`, `backend/api/jobs.py:80-153` (submit-time cache already trusts the
md5), `backend/api/job_worker.py:122-193` (`_job_to_request`/`_run_claimed_job`),
`backend/compute/cloud_run.py` (dispatch + bucket-poll), `backend/config/placement.py` (#521
`resolve_phase_placement`), `backend/worker/__main__.py` (worker validation + `--skip-
`validation`).

```

35. assistant (2026-07-08T06:32:48.501Z)

Review complete. The dimension is largely clean; findings are all low/nit. Here is my assessment.

****Verified clean:****

- ****Config knob****: ``trust_submit_video_id: bool = True`` (models.py:493) is placed idiomatically at the tail of ``DeploymentConfig`` mirroring ``remote_poll_max_wait_sec``, with a comment block matching the surrounding density, and is read via ``getattr(getattr(config,"deployment",None),"trust_submit_video_id",True)`` ? the same defensive pattern as ``compute_target`/`remote_poll_max_wait_sec`` elsewhere, with the fallback default matching the field default.
- ****Shape-pinning tests****: both full-dict assertions in ``test_config_deployment.py`` (lines 68-74 and 405-411) were updated. No other shape-pinning test needs it ? ``test_profile_literal_matches_presets`/`test_compute_target_literal_matches_presets`` only pin the profile Literal ? ``PROFILE_PRESETS`` ? ``PROFILES`` (unaffected by a new bool field), and there is no golden config JSON fixture that enumerates the deployment section.
- ****Tests**** (``test_video_id_trust.py``): fully-mocked/offline (monkeypatch + temp-file DB + ``_HashSpy``), and they genuinely PROVE the win ? ``spy.calls == 0`` on trust, ``== 1`` on every fallback. All required cases present: flag-off, malformed (loops ``not-a-valid-id``, len-31, uppercase, empty), no-id, and both wiring cases (job video_id ? trusted, absent ? None). Ran locally: 42 passed.
- ****Guardrails****: no paid-LLM / licensing / minors surface touched. Confirmed none.
- ****Scope****: exactly the 7 intended files, no stray/untracked files in the worktree, no ``git add -A`` leakage.

****Findings**** (all low/nit):

1. The new doc's own ``[verified]`` code anchors are stale *within this same PR*, because the PR shifts those very lines. ``L4_PRIMARY_TOPOLOGY.md:23`` cites ``orchestration.py:748`` as ``acquire_local_input`` and ``:751`` as the ``compute_video_id`` re-hash, and ``:218`` cites ``:741-763`` (re-hash). Post-change, line 748 is the compat shim, 751 is ``req = db``, and 741-763 is the docstring/import/shim. Actual: ``acquire_local_input`` at :771, re-hash at :798.
2. Merge-conflict surface with #521: the field is appended at the tail of ``DeploymentConfig`` (models.py:493, immediately before ``class BillingConfig`` at :496) ? the natural spot #521 also extends. Semantically non-overlapping, but expect a trivial textual conflict resolvable by keeping both.
3. ``DOC_STATUS.md`` §3 gets no row/sub-note for the new DRAFT design doc. Skipping the §2 archival due-diligence is correctly justified (not a terminal-state flip), but a new tracked design doc idiomatically earns a register line.

36. user (2026-07-08T06:33:01.983Z)

Structured output provided successfully